

Introduction of Randomized Algorithms

COMPSCI 753

Zijian Huang

The University of Auckland

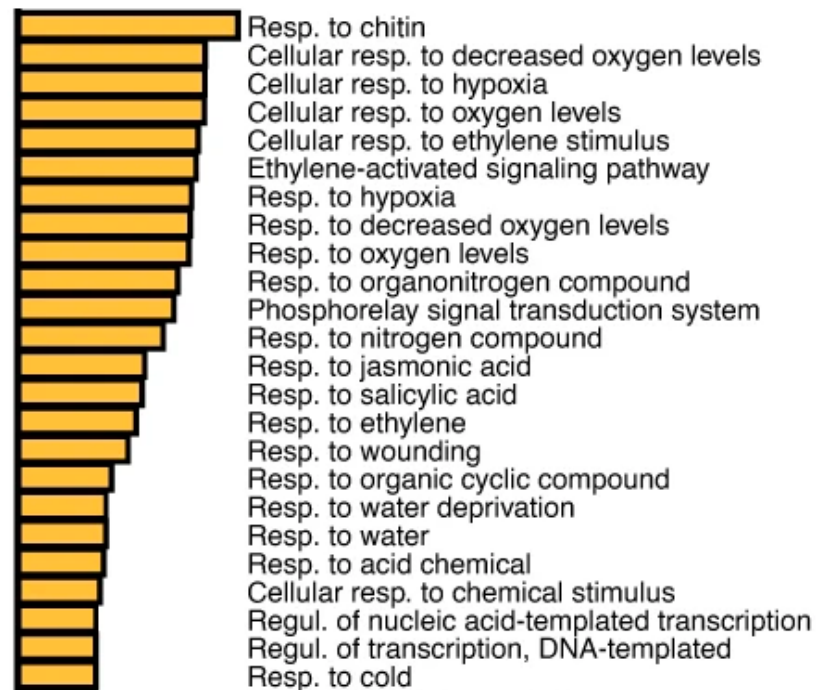


THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

Outline

- Algorithmic Design Patterns
 - Greedy
 - Divide-and-conquer
 - Dynamic programming
 - Network flow
 - Randomization
- Randomized algorithms for big data analytics

High-mutation probability genes



Monroe et al. “Mutation bias reflects natural selection in *Arabidopsis thaliana*”, *Nature*, Jan. 2022

Randomness in the Universe

- Randomness to some degree plays an important role on modeling and analyzing the nature, e.g., random occurrence of mutations with respect to their consequences.

Mutation bias reflects natural selection in *Arabidopsis thaliana*

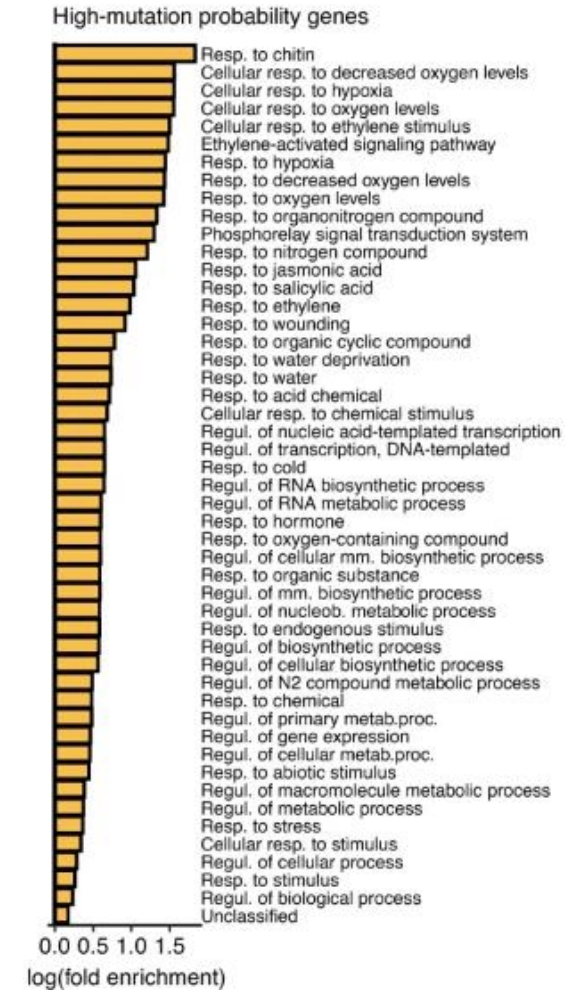
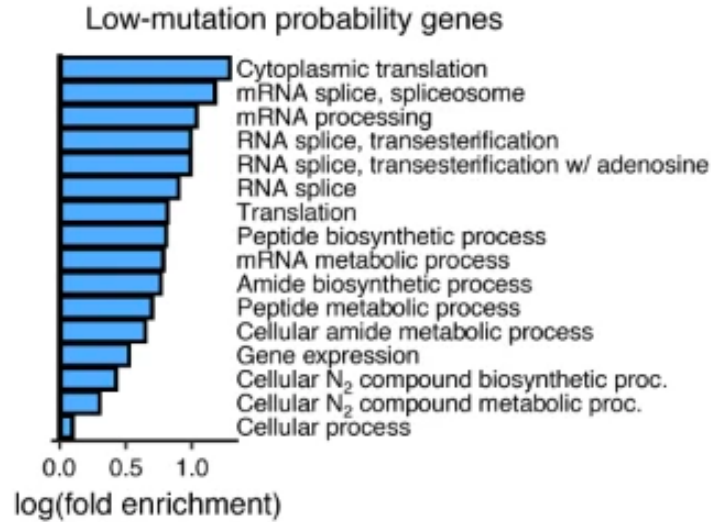
[J. Grey Monroe](#) ✉, [Thanvi Srikant](#), [Pablo Carbonell-Bejerano](#), [Claude Becker](#), [Mariele Lensink](#), [Moises Exposito-Alonso](#), [Marie Klein](#), [Julia Hildebrandt](#), [Manuela Neumann](#), [Daniel Kliebenstein](#), [Mao-Lun Weng](#), [Eric Imbert](#), [Jon Ågren](#), [Matthew T. Rutter](#), [Charles B. Fenster](#) & [Detlef Weigel](#) ✉

[Nature](#) **602**, 101–105 (2022) | [Cite this article](#)

132k Accesses | **28** Citations | **1181** Altmetric | [Metrics](#)

Monroe et al. “Mutation bias reflects natural selection in *Arabidopsis thaliana*”, *Nature*, Jan. 2022

Example: Random Mutation



Monroe et al. “Mutation bias reflects natural selection in *Arabidopsis thaliana*”, *Nature*, Jan. 2022

Randomness in Computer Science

- Randomized models for deterministic objects:
 - Google search engine: modeling web ranking by random walk
- Machine learning theory:
 - Training and testing data should be generated by the same probability distribution
- Statistics via sampling:
 - Election polls
 - Randomized algorithms
 - Randomness speeds up computations

Randomness in Algorithms

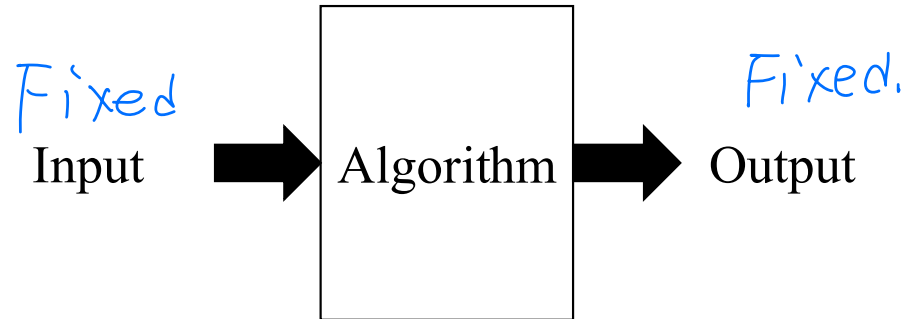
- Algorithm whose performance (or output) depends on the results of random coin flips. It is introduced to avoid the worst-case of algorithms.
- Familiar examples
 - Quicksort
 - Quickselect
- More classic examples
 - Universal hashing
 - Global min cut

Randomness in Algorithms

- General problems:
 - Design: How can we use randomness on designing algorithms to speed up computation?
 - Analyze: How can we guarantee the correctness of results?
- Randomized algorithms are class of algorithms:
 - Making some **random choices** (e.g., flipping a coin) as **part of its logic**.
 - NOT for inputs are chosen randomly.

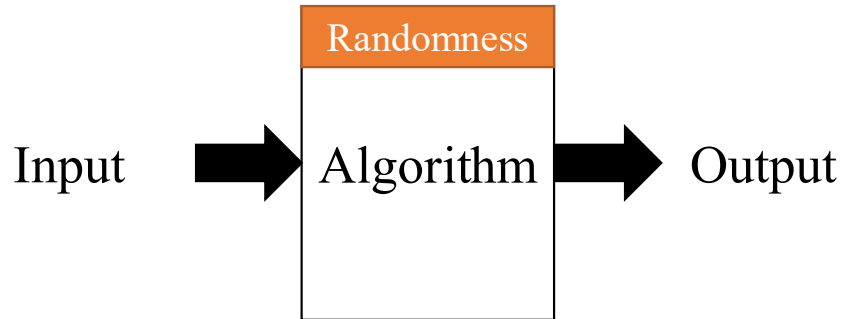
Deterministic Algorithms

- The output and running time of algorithm are functions of only inputs.
- Example: training SGD with fixed input order, divide-and-conquer algorithms, etc...



Randomized Algorithms

- The output and running time of algorithm are functions of both inputs and random bits chosen.
- Example of random bits: flipping a coin i.e., Bernoulli(0.5), random integer i.e., `randint( $n$ )`, ...



Deterministic vs. Randomized

- For any fixed input $x = 3$, what is the output and the running time of these algorithms below:

Deterministic(x)

```
1. y=1
2. If y==0
    While x>0
        x=x-1
3. Return x+y
```

Output: 4

Running Time: $\Theta(1)$

Randomized(x)

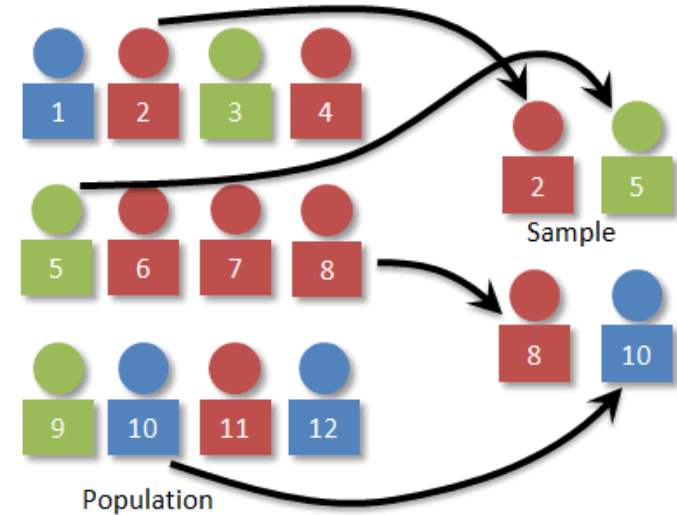
```
1. y=Bernoulli(0.5)
2. If y==0
    While x>0
        x=x-1
3. Return x+y
```

Output: vary

Running Time: vary

Deterministic vs. Randomized

- Deterministic algorithms:
 - Always correct, **AND**
 - Always runs in time $\Theta(T(n))$
- Randomized algorithms:
 - Not always correct, **OR**
 - Not always run in time $\Theta(T(n))$
 - Gamble with correctness or running time
 - Usage: **speed-accuracy trade-off**



Types of Randomized Algorithms

- Problem: Given an array of n elements $A[0, \dots, n - 1]$ where n is even. Half of elements are 0s, the other half are 1s.
- Goal: Find an index containing 1.

Two solutions:

```
Repeat
  1.  $k = \text{randint}(n)$ 
  2. If  $A[k] == 1$ :
      Return  $k$ 
```

Not gamble with correctness
Gamble the running time

```
1. Repeat 100 times
   A.  $k = \text{randint}(n)$ 
   B. If  $A[k] == 1$ :
       Return  $k$ 
2. Return "Failed"
```

Gamble with correctness
Not gamble with the running time

Las Vegas Algorithm

```
Repeat
  1. k=randin(n)
  2. If A[k]==1:
      Return k
```

Not gamble with correctness
Gamble the running time

- Probability of failure: 0
- Worst-case running time: Cannot bound
- Expected running time: $O(1)$ (2 iterations) → Next Lecture!

Monte Carlo Algorithm

```
1.Repeat 100 times
  A. k=randin(n)
  B. If A[k]==1:
      Return k
2.Return "Failed"
```

Gamble with correctness
Not gamble with the running time

- Probability of failure at the 1st time: $\frac{1}{2}$
- Probability of failure of all tries: $\left(\frac{1}{2}\right)^{100}$
- Worst-case running time: $O(1)$

Types of Randomized Algorithms

- Problem: Given an array of n elements $A[0, \dots, n - 1]$ where n is even. Half of elements are 0s, the other half are 1s.
- Goal: Find an index containing 1.

Two solutions:

	Correctness	Runtime
Deterministic	always	$O(n)$
Monte Carlo	with high probability	$O(1)$
Las Vegas	always	$O(1)$ with high probability

Randomized Algorithms: Advantage

- Randomized algorithms are often **simpler, faster, easier to program and parallelize** than their deterministic counterparts
 - Randomized QuickSort is the fastest sorting algorithm in practice

```
RandomizedQuickSort(L, lo, hi)
```

```
  If lo < hi then
```

```
    1. p = RandomPartition(L, lo, hi)
```

```
    2. RandomizedQuickSort(L, lo, p - 1)
```

```
    3. RandomizedQuickSort(L, p+1, hi)
```

// randomly choose pivot

Why randomized algorithms popular?

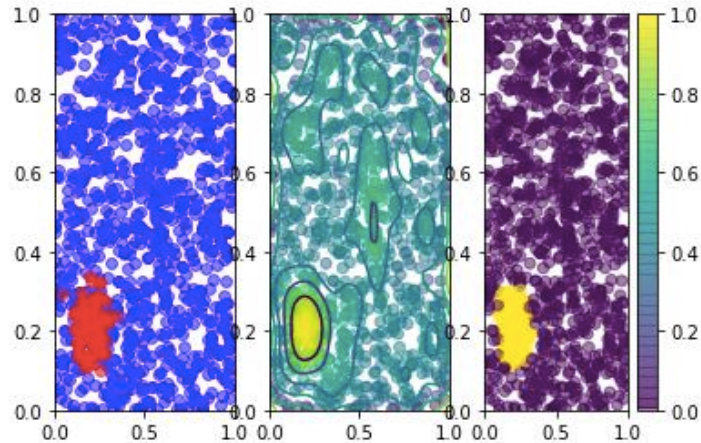
- Study at Microsoft 2008
 - “Cycles, Cells and Platters: An Empirical Analysis of **Hardware Failures** on a Million Consumer PCs.” by Edmund B. Nightingale, John R. Douceur, Vince Orgovan

Event	Probability
The desktop crashes during this lecture	$> 10^{-7}$
RandomizedQuickSort takes time at least double the average when $n \geq 10^6$	$< 10^{-15}$

Efficiency

Why randomized algorithms popular?

- In many applications, randomized algorithms often achieve the “best” 4 –dimensional tradeoff of **time**, **space**, **correctness** (error probability), and programmer **time**



Simplicity

2D Gaussian with Substantial Noise

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import mixture
from sklearn.neighbors import KernelDensity

# Generate 2D Gaussian data
N_c = 100
xy_c = np.random.normal([.2, .2], .05, (N_c, 2))

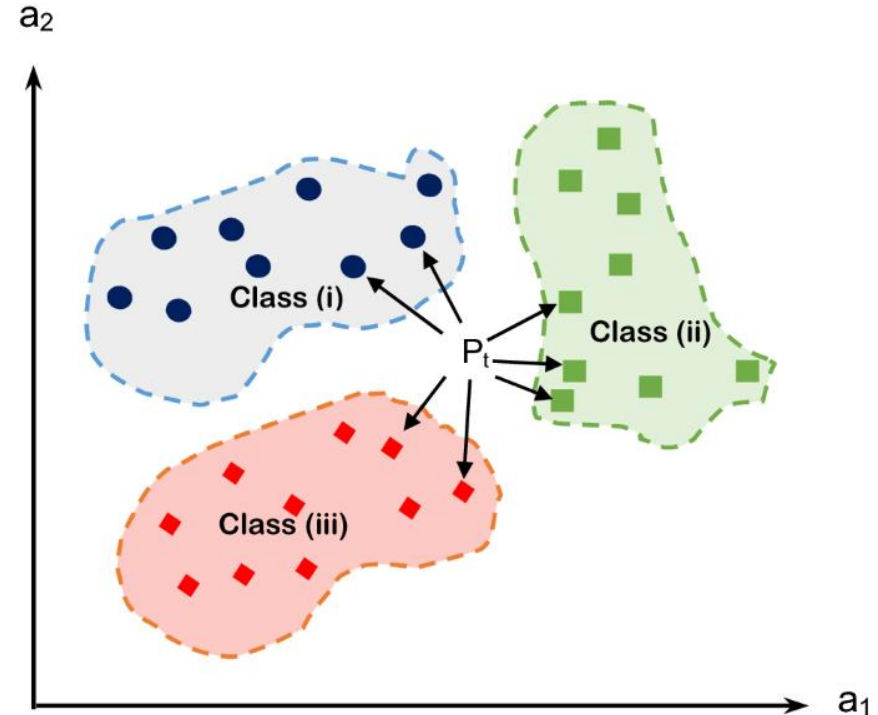
# Generate uniform noise
N_n = 1000
xy_n = np.random.uniform(.0, 1., (N_n, 2))

# Combine into a single data set
data = np.concatenate([xy_c, xy_n])
print(data.shape)

model = KernelDensity(kernel='gaussian', bandwidth=0.05)
model.fit(data)
probs = model.score_samples(data)
```

Outline

- Algorithmic Design Patterns
 - Greedy
 - Divide-and-conquer
 - Dynamic programming
 - Network flow
 - Randomization
- Randomized algorithms for Massive Data analytics
 - Data Stream Algorithms
 - Similarity Search in High Dimensional Space



Data Stream Algorithms

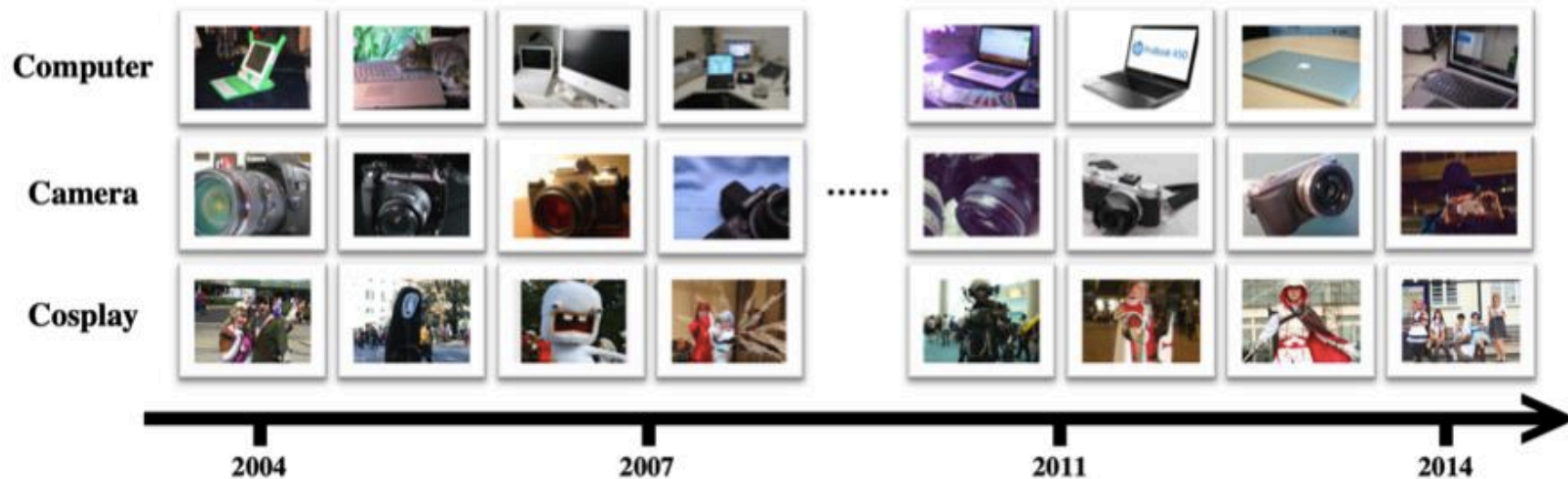
- Data stream analytics tasks:
 - We do not know entire data set in advance.
 - Data arrives in a stream/streams and if it is not processed immediately, it will be lost forever.
- Streaming data and real-time analytics matter more than ever



Data Stream Applications

- Image Stream

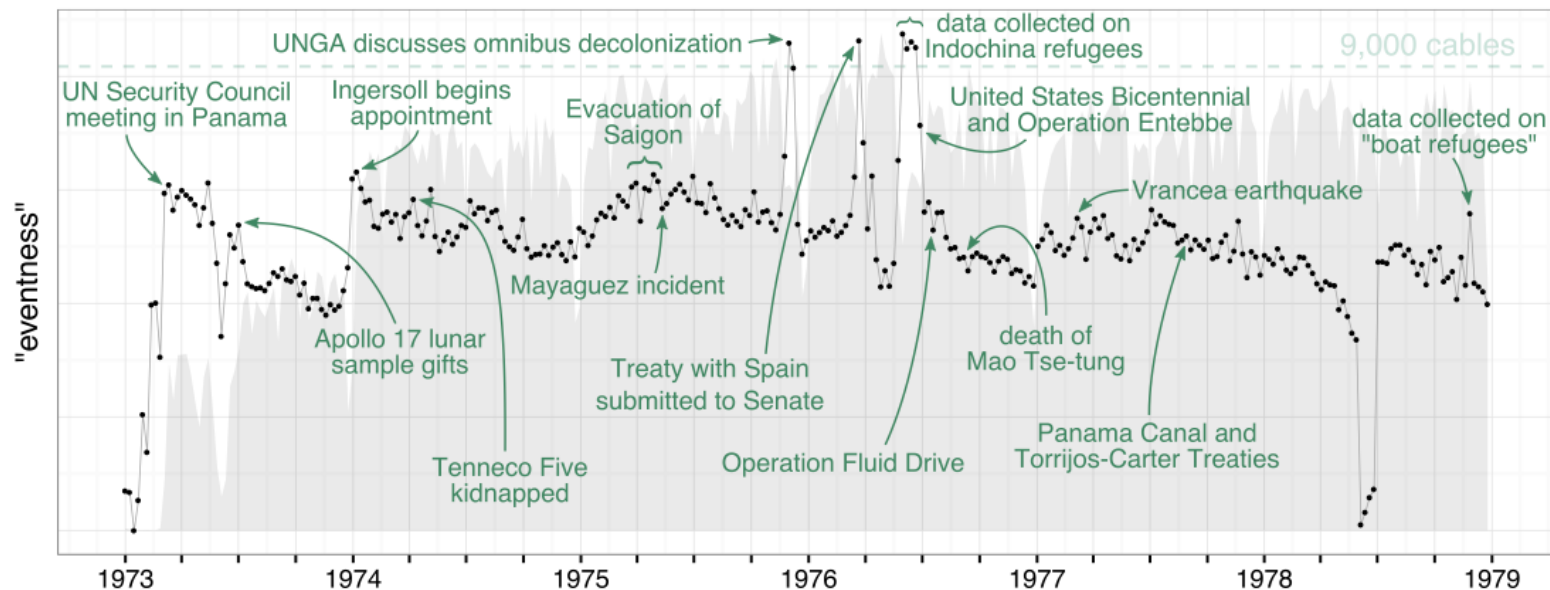
Temporal Evolution of Visual Concepts in Internet Images



Lin et al. "The CLEAR Benchmark: Continual LEARNING on Real-World Imagery", NeurIPS, 2021

Data Stream Applications

- Event Detection on message corpus of over two million U.S. State Department cables



Chaney et al. "Detecting and Characterizing Events", EMNLP, 2016

Data Stream Challenge

- Data stream characteristics
 - Stream elements enter at a rapid rate and the system cannot store the entire stream accessibly
 - Stream data is infinite and non-stationary (the distribution changes over time)
- How do we process the stream using a **limited amount of memory** to answer queries in **real-time** manner?
 - Exact solutions are often impossible without entirely storing data
 - Randomized algorithms provide **approximation answers** with theoretical guarantees on error bounds

Similarity Search in High Dimensional Space

- Metaphor: We can present many problems as finding “similar” items or finding near-neighbors in high dimensions
- Content-based image retrieval:
 - Challenge: Given a database of 1 billion images, find the most similar images to the query in 1ms
 - Google images search counted for 22.6% search queries, i.e. $\approx 15,000$ queries per second

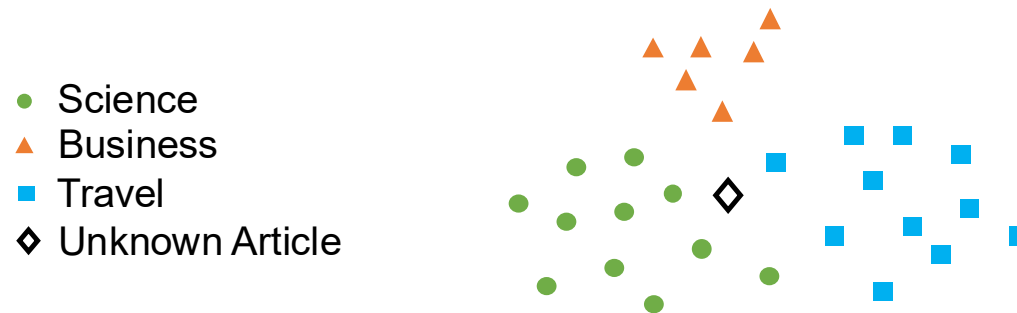


Microsoft Edge

Web browser :

Similarity Search in High Dimensional Space

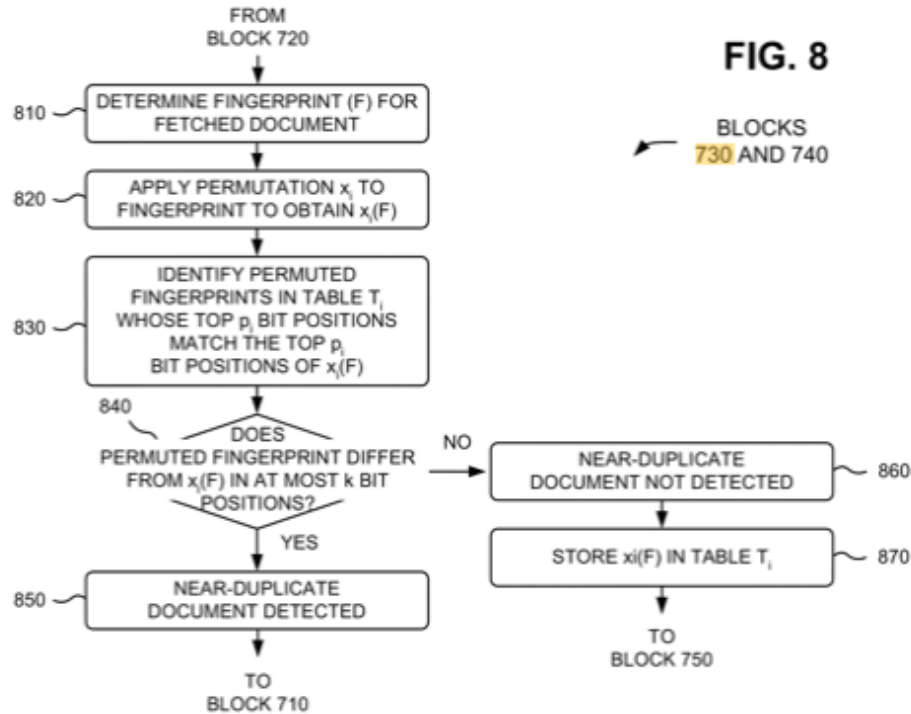
- We can present many problems as finding “similar” items or **finding near-neighbors in high dimensions**
- In machine learning, k -nearest neighbor models for classification, regression, and outlier detection tasks
 - E.g., News Article Classification (3-way Classification, i.e., $n = 3$)



Similarity Search Challenge

- Exact solution - Linear search:
 - Compute the similarity of the query and **EACH** data record $\rightarrow O(nd)$
 - Pick the top-K data records with the highest similarity
- How to scale up similarity search for massive data?
 - Exact solutions are not practical for large-scale data sets and query sets.
 - Randomized algorithms provide **approximation answers** with theoretical guarantees on error bounds.

Near Duplicate Detection



- Flag a fetched document is near-duplicate of a previously fetched document



Exact Solution for Near Duplicate

Given $n = 1$ million documents (10^6),

- A naïve solution needs to compute $n \frac{(n-1)}{2} \sim 5 \times 10^{11}$ pairs of similarities.
- A standard PC computes 106 similarities/sec, then we need approximate 5 days.

Nowadays, the internet has $n \approx 4.5$ billion pages,

- Google used SimHash for approximately detecting web duplicate in their search engine [Manku et al.'07].

Summary

- Algorithms
 - Deterministic Algorithms
 - Randomized Algorithms
- Randomized Algorithms Applications
 - Similarity Search in High Dimensional Space (Week 4-6)
 - Data Stream Algorithms (Week 7-9)

